Technical Report on a Car Dealership Search and Review System

Capstone Project | 10 November 2026
Author: Savzikhanov Ildar | Group: SE-2426

Abstract

This report describes an educational web application for finding car dealerships and publishing reviews. The application extends an IBM Full Stack capstone starter with changes to startup safety, static-file delivery and local execution. Its components were exercised in an isolated environment: nine direct HTTP checks and ten Docker Compose HTTP checks passed, and selected user flows were verified in a browser. A separate check confirmed that a created review remained present after a repeat seed and API restart. The results establish the observed behaviour of this test setup; they do not establish production readiness.

1. Problem and objective

The application brings dealership discovery and review submission into one interface. A visitor can browse dealers, while a registered user can authenticate and contribute a review. The project objective was to make the supplied educational application reproducible and verify the core interactions across its components. No study of market demand, usability with real customers or business impact was performed.

2. Verification question and success criteria

Question. Can the locally configured system support its selected user journeys and preserve a newly created review when its database is seeded again and the API is restarted?

Success criteria. Registration and login succeed; dealer listings and a dealer detail page are accessible; an authenticated user can submit a review and read it back; the added record remains available after the repeat seed and API restart. A successful test requires observed results, not merely started processes.

3. Background and sources

The starting point is the IBM Full Stack Software Developer capstone starter and course assignment [1]. This report identifies changes made in a separate working copy, including safe startup and static-file configuration. The application is a course-derived system, not a claim of an independently designed production product.

Django deployment guidance [2] motivates explicit review of development and deployment settings. OWASP ASVS [3] provides a basis for selecting web security requirements and future verification, while NIST SSDF [4] supplies a framework for integrating secure development and testing into the lifecycle. These references guide further work; the checks in this report do not demonstrate full compliance with them.

4. System architecture

The React client presents account, dealer and review interfaces. Django provides the web application, account actions and car models. For dealer and review data, Django calls a Node.js/Express API backed by MongoDB. A separate Flask service applies the VADER heuristic to review text. Docker Compose defines the local multi-service environment.

Component	Role	Representative source
React and Django	Interface, authentication, routes and car models	server/frontend/src; server/djangoapp
Express and MongoDB	Dealer and review endpoints and persistent data	server/database/app.js; review.js; dealership.js
Flask and VADER	Heuristic review sentiment analysis	server/djangoapp/microservices/app.py
Compose	Local orchestration of application components	compose.yaml

The architecture introduces service boundaries and dependencies. If the API, database or sentiment component is unavailable, the application may not complete the dependent operation. The implementation and tests are limited to the demonstrated local environment.

5. Implementation and changes

The original course project was copied into a separate working directory. WhiteNoise was added to the Python requirements and the handling of React build assets was corrected after JavaScript and CSS returned 404 responses. This makes the tested local application serve the built client assets through Django.

In the original Express startup path, sample-data initialization deleted dealer and review collections. The revised application starts by connecting to MongoDB without deleting or seeding records. Seed data are loaded only through an explicit command; the seed inserts sample data into empty collections and does not clear existing records. This matters when an application is restarted or when a demonstration database already contains a submitted review.

The implementation uses configurable local service addresses, an isolated test database and a documented Docker Compose workflow. The configuration supports a controlled demonstration; it is not a hardened deployment recipe.

6. Method

Source code and local configuration were reviewed first. Functional checks were then performed using real HTTP requests through Django, Express, an isolated MongoDB instance and Flask, without mocked upstream services in the live runs. A separate Docker Compose run exercised the four components together. The test inputs and accounts were synthetic.

The browser walkthrough covered registration, logout and login, the dealer list and review submission and display. A separate restart/seed check counted records and looked up the newly added review after a second seed and an Express restart. Six additional Django tests used mocked upstream calls and are reported separately from the live checks.

The preserved evidence is located under **evidence/** in the project source package. HTTP results, browser observations support different claims and should not be treated as interchangeable.

7. Results

On 10 November 2026, the isolated local run passed all nine recorded HTTP checks. A separate Docker Compose run passed ten recorded checks, including delivery of the frontend JavaScript asset. Each recorded HTTP check returned status 200. The table below describes the scope rather than implying that every possible path was tested.

Verification	Observed result	Evidence
Live HTTP run	9/9 checks passed: home, frontend, registration, logout/login, dealers, dealer detail, review insertion and reading with sentiment	live-http-2026-11-10.json
Docker Compose	10/10 checks passed; includes the frontend JavaScript asset and integrated service paths	compose-http-2026-11-10.json
Browser walkthrough	Registration, logout/login, 50-dealer list, submission and display of a synthetic review were observed	browser-ui-2026-11-10.json
Restart and seed	50 dealers and 51 reviews remained; the added test review was preserved after a repeat seed and API restart	restart-seed-safety-2026-11-10.json
Django checks	Six tests passed with mocked upstream calls; these are separate from the live HTTP checks	django-tests-2026-11-10.txt

The verification criteria were met for the selected synthetic-data scenarios. The record counts are observations from one isolated run, not a performance benchmark or a measurement of a real dealership population.

8. Limitations and future improvements

The checks cover selected local paths. They do not establish cross-browser compatibility, concurrent-write behaviour, failure recovery under load, security of all roles, or production reliability. The VADER score is a heuristic and was not validated against human sentiment labels. No customer interviews, live-user evaluation or statistical effectiveness study was conducted.

Several concrete security and reliability issues remain: some endpoints are marked **csrf_exempt**; the generated fallback Django secret is for development; and assigning review IDs from the current maximum can race under concurrent writes. Future work should address those issues, strengthen authorization and error handling, add broader browser and integration tests, and test load and failure scenarios in an isolated environment.

9. Conclusion

The adapted IBM starter supports the selected local dealership and review journeys. The review-persistence check passed after a repeat seed and API restart. Together, the HTTP, Compose and browser evidence demonstrate those behaviours within the stated test setup, while leaving the security and operational limitations above unresolved.

References

[1] IBM. Full Stack Application Development Capstone Project. Course and starter assignment. https://www.coursera.org/learn/ibm-cloud-native-full-stack-development-capstone

[2] Django Software Foundation. Deployment checklist. Used to frame review of deployment and security settings. https://docs.djangoproject.com/en/stable/howto/deployment/checklist/

[3] OWASP Foundation. Application Security Verification Standard. Used to frame future web security requirements and testing. https://owasp.org/projects/asvs/

[4] NIST. Secure Software Development Framework, SP 800-218 (2022). Used as lifecycle guidance for secure development and verification. https://doi.org/10.6028/NIST.SP.800-218